

Latent Diffusion Models: Supplementary Document - Missing Points

Based on Source Materials

September 7, 2026

Abstract

This is a SUPPLEMENTARY document covering ONLY the points that were MISSED in the previous LDM LaTeX report. This document does NOT repeat any content from the previous report. It covers all missing points including: perceptual loss with $\phi(x)$, adversarial loss, KL regularization, VQ regularization with codebook and stop-gradient, the training pipeline step-by-step, generation pipeline, cross-attention derivation with Q/K/V, self-attention vs cross-attention, concatenation alternative, text-to-image example, semantic compression, high-resolution generation, why not use Transformer directly on pixels, role of convolution, complete mathematical pipeline Stages 1-6, what is learned in each stage table, what latent is NOT, why decoder is important, near-optimal point, why diffusion is suited to latent space, why better than VQGAN+Transformer, image-to-image tasks, general-purpose conditioning, training vs inference, the entire paper in one mathematical chain, deepest intuition, final mental model, L2 blur explanation, VQ loss with stop-gradient, semantic mapping vs semantic synthesis, inpainting, super-resolution, conditional latent diffusion, Transformer encoders, convolutional sampling beyond 256^2 , perceptual compression tradeoffs, complete connection between sections diagram, one-line meaning of each section table, the big picture, ELBO derivation with equations (4), (8), (9), (10), (13), (14), Markov relationship, reverse process, $x_\theta(x_t, t)$ definition, $\epsilon_\theta(x_t, t)$ equation (14), crucial transformation, SNR relationship, whole derivation in one map, complete LDM training objective, generation, sampling definition, text $\rightarrow$ tokens $\rightarrow$ embeddings $\rightarrow$ Transformer pipeline, cross-attention inside U-Net, image-conditioned diffusion, vision encoder, condition modalities, and all remaining points.

Contents

1	The "Don't Model Every Pixel Directly" Philosophy	8
2	The $48\times$ Reduction	8
3	Why Not Compress Much More?	8
4	The Key Insight: "Moderate Compression + Diffusion's Spatial Inductive Bias"	9
5	Perceptual Loss	9

6	Adversarial Loss	9
7	KL Regularization	10
8	VQ Regularization	10
9	VQ Definition and VQ Loss	10
9.1	VQ Loss	11
9.2	Important Point About the LDM Paper	11
10	Why Spatial Structure Matters So Much	12
11	Training Pipeline - Step by Step	12
11.1	Step 1 — Encode	12
11.2	Step 2 — Randomly select a timestep	12
11.3	Step 3 — Sample Gaussian noise	12
11.4	Step 4 — Add the appropriate amount of noise	12
11.5	Step 5 — Feed it to UNet	12
11.6	Step 6 — Compare predicted and true noise	12
11.7	Step 7 — Backpropagation	12
12	Generation Pipeline - Step by Step	13
13	Why is Decoding Cheap?	13
14	Title Breakdown	13
15	Figure 2 Explanation	14
15.1	Perceptual compression	14
15.2	Semantic compression	14
16	The "Two Specialists" Analogy	14
16.1	Model A — Autoencoder	14
16.2	Model B — Diffusion	15
17	Why Not Train Everything Jointly?	15
17.1	Phase 1	15
17.2	Phase 2	15
18	Conditioning via Cross-Attention - Complete Derivation	16
18.1	Text Encoding	16
18.2	UNet Representation	16
18.3	Query, Key, Value for Cross-Attention	17
19	Self-Attention vs Cross-Attention	17
19.1	Self-Attention	17
19.2	Cross-Attention	17
20	The Attention Equation	18
21	Example: "A Red Car in Snow"	18

22 Why Cross-Attention is Better Than Concatenating Text	18
22.1 Concatenation	19
22.2 Cross-Attention	19
23 The Concatenation Alternative	19
23.1 The Limitation of Concatenation	19
24 Text-to-Image Example: "A Golden Retriever Running on a Beach at Sunset"	20
24.1 Step 1 — Text encoding	20
24.2 Step 2 — Start with random latent	20
24.3 Step 3 — First denoising step	20
24.4 Step 4 — Cross-attention	20
24.5 Step 5 — Update latent	21
24.6 Step 6 — Repeat	21
24.7 Step 7 — Decode	21
25 Important Distinction: Text Does NOT Generate Image Directly	21
26 What Does the UNet Actually Learn?	21
27 Why Does Denoising Create Semantic Structure?	22
28 Paper's Equation (2)	22
29 Paper's Equation (3)	23
30 Semantic Compression - Detailed Explanation	23
31 Why This Improves High-Resolution Generation	24
32 Why Can It Generate Megapixel Images Convolutionally?	24
33 Why Not Use a Transformer Directly on Pixels?	24
34 The Role of Convolution	25
35 Full Mathematical Pipeline - Stages 1-6	25
35.1 Stage 1: Autoencoder	25
35.2 Stage 2: Forward latent diffusion	25
35.3 Stage 3: Train latent denoiser	25
35.4 Stage 4: Conditional version	26
35.5 Stage 5: Reverse diffusion	26
35.6 Stage 6: Decode	26
36 One Diagram to Memorize	26
37 What is Actually Learned in Each Stage?	27
38 What the Latent is NOT	28

39 Why the Decoder is So Important	28
39.1 Case A	29
39.2 Case B	29
40 Why "Less Aggressive Downsampling" Matters	29
41 Why the Authors Say "Near-Optimal Point"	29
41.1 Computational complexity	29
41.2 Reconstruction quality	30
42 Why Diffusion is Particularly Suited to the Latent Space	30
43 Why This is Better Than the Earlier VQGAN + Transformer Idea	30
44 How Image-to-Image Tasks Fit Into This	31
44.1 Inpainting	31
44.2 Super-resolution	31
44.3 Semantic synthesis	31
44.4 Text-to-image	31
45 Why Cross-Attention is Called "General-Purpose Conditioning"	31
45.1 Text	32
45.2 Semantic map	32
45.3 Bounding boxes	32
46 One Subtle Point	32
47 Training Versus Inference	32
47.1 Training	32
47.2 Inference	33
48 Why This Was Such an Important Paper	33
49 The Entire Paper in One Mathematical Chain	33
50 The Deepest Intuition	34
51 Final Mental Model	34
51.1 Brain 1 — Encoder	34
51.2 Brain 2 — Diffusion UNet	34
51.3 Brain 3 — Decoder	34
51.4 Controller — Text encoder + cross-attention	34
52 Why L2 Creates Blur (The Autoencoder Problem)	35
53 What the LDM Paper Does Instead	35
54 Model B — The Diffusion Model Takes Over	36
54.1 What It Learns	36
54.2 What Does the Neural Network Actually Learn?	36

55 The Whole LDM Pipeline	37
56 But "Accurately" Has Two Meanings	37
57 Training is Conceptually Done in Two Stages	38
57.1 Stage 1 — Train the Autoencoder	38
57.2 Stage 2 — Train the Diffusion Model	38
57.3 During Generation	39
58 Complete Training Map	39
59 Complete Generation Map	40
60 The Simplest Mental Model	41
60.1 Stage 1 — Autoencoder	41
60.2 Stage 2 — Diffusion	41
60.3 Final System	42
61 Text Conditioning During Training AND Generation	42
61.1 During Training	42
61.2 During Generation	43
61.3 One Important Distinction	43
62 Semantic Mapping vs Semantic Synthesis	44
62.1 What is a Semantic Map?	44
62.2 Semantic Mapping	44
62.3 Semantic Synthesis	45
62.4 The Difference in One Line	45
62.5 Why is This Relevant to the LDM Paper?	46
62.6 What Happens Internally?	46
62.7 Why Does the Semantic Map Not Specify the Exact Pixels?	47
62.8 Semantic Synthesis is Therefore a Conditional Distribution	47
62.9 Example: Semantic Synthesis	48
62.10 Semantic Mapping vs Semantic Synthesis in the LDM Context	48
62.11 The Easiest Way to Remember It	49
63 Applications of Latent Diffusion Models	49
63.1 Inpainting with Latent Diffusion	49
63.1.1 What is Inpainting?	49
63.1.2 How LDM Does This	50
63.1.3 The Important Idea	50
63.1.4 Latent-Space Version	50
63.1.5 Why Latent Diffusion Helps	51
63.2 Super-Resolution with Latent Diffusion	51
63.2.1 Conditional Diffusion	51
63.2.2 Why This is Different from Simply Upscaling	52
63.3 Conditional Latent Diffusion	52
63.3.1 How Conditioning Enters the Model	53
63.4 Transformer Encoders for LDM	53
63.5 Convolutional Sampling Beyond 256^2	54

63.5.1	Why Convolution Helps	54
63.6	Image Generation with Latent Diffusion	55
63.6.1	Training	55
63.6.2	Generation	55
63.7	On Perceptual Compression Tradeoffs	56
63.7.1	Too Little Compression	56
63.7.2	Too Much Compression	56
63.7.3	The Sweet Spot	56
63.8	Why Pixel-Wise MSE Matters Here	57
64	The Complete Connection Between All These Sections	57
65	One-Line Meaning of Each Section	59
66	The Big Picture	59
67	Mathematical Derivation of the Diffusion Objective	59
67.1	Forward Diffusion Starts from x_0	59
67.2	Markov Relationship Between Two Time Steps	60
67.3	Now Reverse the Process	60
67.4	Why Do We Need ELBO?	61
67.5	The Key Problem: Reverse Process Needs x_0	61
67.6	The Reverse Step Becomes Gaussian	62
67.7	ELBO Becomes an x_0 -Prediction Objective	62
67.8	Instead of Predicting x_0 , Predict the Noise	62
67.9	Why This Works	63
67.10	The Crucial Transformation	63
67.11	The Whole Derivation in One Map	63
67.12	During Generation	64
67.13	And for Latent Diffusion Specifically	64
68	KL Divergence Between Gaussians	65
68.1	Setup	65
68.2	KL Definition	65
68.3	Gaussian Densities	65
68.4	Take the Ratio	66
68.5	Take the Log	66
68.6	Take Expectation Under q	66
68.7	Substitute	66
68.8	Application to Diffusion	67
69	Sampling in LDM	67
69.1	During Training	67
69.2	During Sampling	67
69.3	In Inpainting Specifically	68
69.4	Why is It Called "Sampling"?	68

70	Text $\rightarrow$ Tokens $\rightarrow$ Embeddings $\rightarrow$ Transformer	69
70.1	Step 1: Text $\rightarrow$ Tokens	69
70.2	Step 2: Tokens $\rightarrow$ Embeddings	69
70.3	Step 3: Transformer Encoder / Self-Attention	69
70.4	Step 4: Meanwhile, the Image/Latent Goes Through Diffusion	70
70.5	Step 5: Now Comes Cross-Attention	70
70.5.1	Query	70
70.5.2	Key and Value	70
70.6	Step 6: Then This Information is Used Inside the U-Net	71
71	Complete Conceptual Pipeline	71
72	The Key Distinction	72
73	Cross-Attention Happens Inside the U-Net	72
74	One Important Distinction	73
75	Image-Conditioned Diffusion	74
75.1	Text-Conditioned Diffusion	74
75.2	Image-Conditioned Diffusion	74
75.3	Complete Architecture with Image Condition	75
75.4	Two Common Ways to Use an Image Condition	75
75.4.1	1. Image as Semantic Conditioning	75
75.4.2	2. Image as Spatial Conditioning	75
75.5	For Your Understanding of LDM	75
76	Summary of Key Equations	76
77	Conclusion	77

1 The "Don't Model Every Pixel Directly" Philosophy

The important idea is:

$$\boxed{\text{Don't model every pixel directly.}} \quad (1)$$

Instead:

$$\boxed{\text{compress} \rightarrow \text{diffuse in latent space} \rightarrow \text{decode}} \quad (2)$$

2 The $48\times$ Reduction

Compare:

$$786,432 \text{ pixel values versus } 16,384 \text{ latent values} \quad (3)$$

That's roughly a $48\times$ reduction in the number of values.

And because the diffusion UNet is repeatedly evaluated during generation, this reduction is extremely important.

3 Why Not Compress Much More?

This is where the paper's Figure 1 becomes important.

They compare different downsampling factors:

$$f = 4, \quad f = 8, \quad f = 16 \quad (4)$$

For example, 512×512 becomes:

$$f = 4 : \quad 128 \times 128 \quad (5)$$

$$f = 8 : \quad 64 \times 64 \quad (6)$$

$$f = 16 : \quad 32 \times 32 \quad (7)$$

Obviously, 32×32 is much cheaper than 128×128 . But there's a problem. More compression means more information loss.

Therefore there is a trade-off:

$$\boxed{\text{compression} \leftrightarrow \text{reconstruction quality}} \quad (8)$$

The paper wants the sweet spot.

4 The Key Insight: "Moderate Compression + Diffusion's Spatial Inductive Bias"

The paper's point is: We can keep more spatial information while still dramatically reducing computation.

This gives:

$$\boxed{\text{moderate compression + diffusion's spatial inductive bias}} \quad (9)$$

instead of:

$$\boxed{\text{extreme compression + huge autoregressive transformer}} \quad (10)$$

5 Perceptual Loss

Instead of asking only: "Are the pixels numerically identical?" we ask: "Does the reconstruction look perceptually similar?"

A perceptual network extracts features:

$$\phi(x) \quad (11)$$

Then a perceptual loss can be something like:

$$L_{\text{perceptual}} = \|\phi(x) - \phi(\tilde{x})\| \quad (12)$$

This encourages the reconstruction to retain meaningful visual structure.

6 Adversarial Loss

The autoencoder also uses a patch-based discriminator.

Conceptually:

$$x \rightarrow \text{real} \quad (13)$$

while:

$$\tilde{x} \rightarrow \text{fake} \quad (14)$$

The discriminator tries to distinguish them. The decoder tries to generate realistic reconstructions. This helps prevent overly smooth reconstructions.

So the autoencoder is not simply:

$$\text{compress} \rightarrow \text{decompress} \quad (15)$$

It is trained to produce:

$$\boxed{\text{compact + perceptually faithful + locally realistic}} \quad (16)$$

representations.

7 KL Regularization

The paper discusses two regularization strategies: KL-reg and VQ-reg.

Let's first understand KL-reg.

The latent shouldn't become some completely uncontrolled distribution. So they add a small KL penalty encouraging the latent distribution to remain reasonably close to a standard Gaussian:

$$\mathcal{N}(0, I) \tag{17}$$

Conceptually:

$$L = L_{\text{reconstruction}} + \lambda_{\text{KL}} L_{\text{KL}} + \lambda_{\text{adv}} L_{\text{adv}} \tag{18}$$

But notice something important: This is not the same as saying LDM is a VAE. The first-stage model borrows ideas from VAEs, particularly KL regularization, but the full LDM consists of:

autoencoder + latent diffusion

(19)

rather than simply being a VAE.

8 VQ Regularization

The other option is vector quantization.

Instead of allowing completely arbitrary latent values, latent representations are associated with learned codebook vectors. This is related to VQ-VAE/VQGAN.

But the authors modify the design so that the diffusion model can work with a continuous two-dimensional latent structure.

This is important because:

$$z \in \mathbb{R}^{h \times w \times c} \tag{20}$$

retains spatial organization.

9 VQ Definition and VQ Loss

VQ = Vector Quantization.

Instead of allowing the encoder to produce completely arbitrary continuous latent vectors, VQ maps each latent vector to the nearest vector in a learned codebook.

Suppose the encoder produces:

$$z_e(x) \tag{21}$$

and the codebook contains:

$$\mathcal{C} = \{e_1, e_2, \dots, e_K\} \tag{22}$$

The quantized latent is:

$$z_q(x) = e_k \tag{23}$$

where:

$$k = \arg \min_j \|z_e(x) - e_j\|_2 \quad (24)$$

So:

$$\boxed{z_e(x) \rightarrow z_q(x)} \quad (25)$$

9.1 VQ Loss

A typical VQ objective contains terms such as:

$$\mathcal{L}_{VQ} = \|\text{sg}[z_e] - e\|_2^2 + \beta \|z_e - \text{sg}[e]\|_2^2 \quad (26)$$

where:

- z_e = encoder output
- e = selected codebook vector
- sg = stop-gradient
- β = commitment-loss coefficient

The first term updates the codebook toward the encoder output. The second encourages the encoder to commit to the selected codebook vector.

9.2 Important Point About the LDM Paper

The LDM paper actually discusses two types of regularization for the first-stage autoencoder:

- KL-reg — a small KL regularization on the latent representation
- VQ-reg — vector quantization of the latent representation

The paper finds that both can work as ways of regularizing the latent space. The important distinction is:

$$\boxed{\text{VQ-reg} \Rightarrow \text{discrete/quantized latent representation}} \quad (27)$$

while:

$$\boxed{\text{KL-reg} \Rightarrow \text{continuous, approximately regularized latent representation}} \quad (28)$$

For the LDM approach, the authors use KL-regularized autoencoders in their main experiments rather than requiring a discrete VQ latent space.

So when you see "VQ-reg" in the paper, think: "Instead of forcing the latent to follow a Gaussian distribution, constrain it through a learned vector-quantization codebook."

And this is separate from the perceptual + adversarial losses, which are primarily there to make the reconstructed image visually convincing rather than blurry.

10 Why Spatial Structure Matters So Much

Imagine the latent:

$$64 \times 64 \times 4 \tag{29}$$

The location (i, j) still corresponds roughly to a spatial region of the original image. So neighboring latent positions correspond to neighboring image regions.

For example, $z_{10,10}$ may contain information associated with some upper-left image region, while $z_{10,11}$ contains information from a nearby region.

This means convolutions can exploit local structure. That’s a huge advantage.

11 Training Pipeline - Step by Step

Let’s walk through one training example.

Suppose we have an image x .

11.1 Step 1 — Encode

$$z_0 = E(x) \tag{30}$$

11.2 Step 2 — Randomly select a timestep

Choose:

$$t \sim U\{1, \dots, T\} \tag{31}$$

For example, $t = 437$.

11.3 Step 3 — Sample Gaussian noise

$$\epsilon \sim \mathcal{N}(0, I) \tag{32}$$

11.4 Step 4 — Add the appropriate amount of noise

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \tag{33}$$

11.5 Step 5 — Feed it to UNet

$$\epsilon_\theta(z_t, t) \tag{34}$$

11.6 Step 6 — Compare predicted and true noise

$$L = \|\epsilon - \epsilon_\theta(z_t, t)\|^2 \tag{35}$$

11.7 Step 7 — Backpropagation

Update the diffusion UNet.

That’s it. The diffusion model doesn’t need to see the full-resolution image. It sees the latent.

12 Generation Pipeline - Step by Step

Now suppose we don't have an image. We want to generate one.

We start with random latent noise:

$$z_T \sim \mathcal{N}(0, I) \quad (36)$$

Then repeatedly denoise:

$$z_T \rightarrow z_{T-1} \rightarrow z_{T-2} \rightarrow \cdots \rightarrow z_0 \quad (37)$$

The UNet predicts noise at every step:

$$\epsilon_\theta(z_t, t) \quad (38)$$

Using that prediction, we calculate the mean of the reverse Gaussian:

$$\mu_\theta(z_t, t) \quad (39)$$

Then:

$$z_{t-1} = \mu_\theta(z_t, t) + \sigma_t \eta \quad (40)$$

where:

$$\eta \sim \mathcal{N}(0, I) \quad (41)$$

This is exactly the same reverse-diffusion logic.

13 Why is Decoding Cheap?

Because the decoder runs only once.

During sampling, suppose we use 50 diffusion steps:

$$z_{50} \rightarrow z_{49} \rightarrow \cdots \rightarrow z_0 \quad (42)$$

The diffusion UNet runs approximately 50 times. But the decoder runs:

$$\boxed{\text{once}} \quad (43)$$

at the end.

Therefore, almost all the iterative computation happens in the compressed latent space.

14 Title Breakdown

The title is: "High-Resolution Image Synthesis with Latent Diffusion Models"

Break it down:

- **High-resolution:** They want images such as 512^2 , 1024^2 , megapixel images.
- **Image synthesis:** Generate new images.

- **Latent:** The diffusion process happens in z instead of x .
- **Diffusion models:** The generation mechanism is still the DDPM-style iterative denoising process.

15 Figure 2 Explanation

The paper says there are two forms of compression.

15.1 Perceptual compression

Remove:

$$\text{imperceptible details} \tag{44}$$

15.2 Semantic compression

Learn:

$$\text{meaningful composition} \tag{45}$$

such as:

- objects
- relationships
- layout
- semantics

The authors want the autoencoder to handle the first kind. Then diffusion handles the second kind.

This separation is one of the deepest ideas in the paper.

16 The "Two Specialists" Analogy

Imagine two models.

16.1 Model A — Autoencoder

Question: "How can I represent this image compactly while still allowing a visually faithful reconstruction?"

It learns:

$$x \leftrightarrow z \tag{46}$$

16.2 Model B — Diffusion

Question: "What kinds of latent representations correspond to realistic images, and how can I generate them?"

It learns:

$$p(z) \tag{47}$$

So:

Autoencoder learns representation

(48)

and:

Diffusion learns distribution

(49)

This separation makes training much easier.

17 Why Not Train Everything Jointly?

The paper emphasizes that they don't need to jointly optimize:

$$E + D + \text{diffusion} \tag{50}$$

from scratch.

Instead:

17.1 Phase 1

Train:

$$E, D \tag{51}$$

17.2 Phase 2

Train diffusion on:

$$z = E(x) \tag{52}$$

This has a major advantage. Once you have a good autoencoder, you can reuse it for multiple generative models.

For example, E, D can support:

- unconditional generation
- text-to-image
- image-to-image
- inpainting
- super-resolution
- other conditional generation tasks

So the expensive representation-learning stage doesn't need to be repeated every time.

18 Conditioning via Cross-Attention - Complete Derivation

So far we've generated $p(z)$. But what if we want $p(z|y)$ where y is a condition such as text?

The diffusion network becomes:

$$\epsilon_\theta(z_t, t, y) \quad (53)$$

The paper implements this through cross-attention.

18.1 Text Encoding

Let the text prompt be y . A text encoder produces:

$$\tau_\theta(y) \quad (54)$$

Suppose the text has M tokens. Then:

$$\tau_\theta(y) \in \mathbb{R}^{M \times d_\tau} \quad (55)$$

So we have a sequence of text embeddings:

$$[e_1, e_2, \dots, e_M] \quad (56)$$

18.2 UNet Representation

At some intermediate UNet layer, suppose the latent feature map is:

$$\phi_i(z_t) \quad (57)$$

Because it is spatial, we can flatten it:

$$\phi_i(z_t) \in \mathbb{R}^{N \times d_i} \quad (58)$$

Here:

- N = number of spatial positions
- d_i = feature dimension

For example, if the feature map is $16 \times 16 \times 320$, then:

$$N = 16 \times 16 = 256 \quad (59)$$

So:

$$\phi_i(z_t) \in \mathbb{R}^{256 \times 320} \quad (60)$$

18.3 Query, Key, Value for Cross-Attention

The UNet representation provides:

$$Q = W_Q \phi_i(z_t) \quad (61)$$

The text provides:

$$K = W_K \tau_\theta(y) \quad (62)$$

and:

$$V = W_V \tau_\theta(y) \quad (63)$$

So:

$$\boxed{Q = \text{image/latent features}} \quad (64)$$

while:

$$\boxed{K, V = \text{text features}} \quad (65)$$

This is why it is called cross-attention.

19 Self-Attention vs Cross-Attention

19.1 Self-Attention

Everything comes from the same sequence.

$$Q = XW_Q \quad (66)$$

$$K = XW_K \quad (67)$$

$$V = XW_V \quad (68)$$

So:

$$X \rightarrow X \quad (69)$$

19.2 Cross-Attention

Q comes from one modality and K/V from another.

Here:

$$Q = \text{UNet features} \quad (70)$$

and:

$$K, V = \text{text} \quad (71)$$

So:

$$\boxed{\text{image features} \rightarrow \text{text information}} \quad (72)$$

20 The Attention Equation

The paper gives:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V \quad (73)$$

Let's understand every part.

Suppose:

$$Q \in \mathbb{R}^{N \times d} \quad (74)$$

and:

$$K \in \mathbb{R}^{M \times d} \quad (75)$$

Then QK^T has dimension:

$$N \times M \quad (76)$$

This is beautiful. It means every spatial position in the image gets an attention score against every text token.

So:

$$N \times M \quad (77)$$

means: For every image feature location, how relevant is each word/token?

21 Example: "A Red Car in Snow"

Prompt: "A red car in snow"

Tokens might roughly correspond to:

$$[\text{red}, \text{car}, \text{snow}] \quad (78)$$

Suppose the image feature at location i corresponds to the car. Its attention might look conceptually like:

$$[0.3, 0.6, 0.1] \quad (79)$$

So the UNet is saying: "For this spatial feature, the word 'car' is particularly relevant." Another location corresponding to the background might have:

$$[0.1, 0.1, 0.8] \quad (80)$$

Meaning: "This location is strongly associated with 'snow'."

This isn't a hard symbolic assignment; it's learned continuous attention.

22 Why Cross-Attention is Better Than Concatenating Text

The paper actually discusses two possibilities.

22.1 Concatenation

You could inject conditioning information directly into the UNet.

For example:

$$[z_t, y] \tag{81}$$

But this is relatively restrictive.

22.2 Cross-Attention

Cross-attention is more flexible. It allows text to interact dynamically with spatial features at multiple locations and resolutions.

23 The Concatenation Alternative

You could instead project the text embedding and concatenate it with the latent features.

For example, if you have a single text embedding $c \in \mathbb{R}^d$, you can broadcast it spatially:

$$c \rightarrow \mathbb{R}^{H \times W \times d} \tag{82}$$

and concatenate:

$$\boxed{z' = [z_t; c]} \tag{83}$$

giving:

$$z' \in \mathbb{R}^{H \times W \times (C+d)} \tag{84}$$

Then a convolution/linear layer processes the combined representation.

23.1 The Limitation of Concatenation

With concatenation, every spatial location initially receives essentially the same text conditioning.

With cross-attention:

$$\boxed{\text{different latent locations can attend to different text tokens}} \tag{85}$$

For a prompt: "A red car beside a blue house", cross-attention can learn relationships such as:

$$\text{car region} \leftrightarrow \text{"red"}, \text{"car"} \tag{86}$$

$$\text{house region} \leftrightarrow \text{"blue"}, \text{"house"} \tag{87}$$

So if you cannot use cross-attention, concatenation is a valid simpler conditioning mechanism, but you generally lose the flexible token-to-spatial-feature interaction that cross-attention provides.

24 Text-to-Image Example: "A Golden Retriever Running on a Beach at Sunset"

Suppose the prompt is: "A golden retriever running on a beach at sunset."

24.1 Step 1 — Text encoding

The prompt becomes token representations:

$$y \rightarrow \tau(y) \tag{88}$$

So we have:

$$\tau(y) = [e_1, e_2, \dots, e_M] \tag{89}$$

24.2 Step 2 — Start with random latent

$$z_T \sim \mathcal{N}(0, I) \tag{90}$$

Notice: We do not start with a noisy 512×512 RGB image. We start with latent noise.

24.3 Step 3 — First denoising step

UNet receives:

$$(z_T, T, \tau(y)) \tag{91}$$

It predicts:

$$\epsilon_\theta(z_T, T, \tau(y)) \tag{92}$$

24.4 Step 4 — Cross-attention

At various UNet layers:

$$Q = W_Q \phi(z_T) \tag{93}$$

and:

$$K = W_K \tau(y) \tag{94}$$

$$V = W_V \tau(y) \tag{95}$$

Attention tells the UNet which parts of the text are relevant to its current spatial features.

24.5 Step 5 — Update latent

Calculate:

$$z_{T-1} \tag{96}$$

24.6 Step 6 — Repeat

$$z_{T-1} \rightarrow z_{T-2} \rightarrow \cdots \rightarrow z_0 \tag{97}$$

At each stage, the text condition continues influencing the denoising.

24.7 Step 7 — Decode

Finally:

$$x = D(z_0) \tag{98}$$

And we obtain:

golden retriever + beach + sunset

(99)

25 Important Distinction: Text Does NOT Generate Image Directly

The text does NOT generate the image directly.

This is a common misunderstanding.

The text encoder doesn't output pixels. Instead:

$$\text{text} \rightarrow \text{text embeddings} \tag{100}$$

and those embeddings condition the denoising process. The diffusion model still generates the visual representation.

So the causal chain is:

text $\rightarrow$ conditioning information $\rightarrow$ guided latent denoising $\rightarrow$ latent image $\rightarrow$ pixels

(101)

26 What Does the UNet Actually Learn?

This is another critical point.

It does not simply learn: "What does a dog look like?"

Instead, it learns a conditional noise prediction function:

$$\epsilon_{\theta}(z_t, t, \tau(y)) \tag{102}$$

For each noisy latent, it learns: given how noisy this latent is, what noise is present, and what text condition should this latent represent, what noise should be removed?

That is fundamentally a denoising problem.

27 Why Does Denoising Create Semantic Structure?

Because the training dataset contains image-text relationships.

Suppose the model repeatedly sees:

$$(\text{"dog"}, \text{dog images}) \tag{103}$$

and:

$$(\text{"car"}, \text{car images}) \tag{104}$$

The model gradually learns how text embeddings correlate with visual latent structures.

Over many examples, the conditional distribution:

$$p(z|y) \tag{105}$$

becomes learned.

Therefore, when you provide $y = \text{"dog on beach"}$, the model samples from the learned conditional distribution.

28 Paper's Equation (2)

The paper writes:

$$L_{\text{LDM}} = \mathbb{E}_{E(x), \epsilon, t} [\|\epsilon - \epsilon_{\theta}(z_t, t)\|_2^2] \tag{106}$$

Let's decode it carefully.

- x : Original image
- $E(x)$: Encoded latent $z_0 = E(x)$
- ϵ : Random Gaussian noise $\epsilon \sim N(0, I)$
- t : Random timestep
- z_t : Noisy latent
- ϵ_{θ} : UNet's prediction

Therefore:

$$\boxed{\text{actual noise} - \text{predicted noise}} \tag{107}$$

is minimized.

29 Paper’s Equation (3)

For conditioning:

$$L_{\text{LDM}} = \mathbb{E} [\|\epsilon - \epsilon_{\theta}(z_t, t, \tau_{\theta}(y))\|_2^2] \quad (108)$$

Now the UNet receives:

$$z_t + t + \tau_{\theta}(y) \quad (109)$$

So the model learns:

predict noise given noisy latent + timestep + condition

 (110)

30 Semantic Compression - Detailed Explanation

This is worth revisiting because the paper uses it in a subtle way.

Suppose an image has:

- exact pixel noise
- tiny texture variations
- edges
- shapes
- objects
- object relationships
- scene meaning

The first-stage autoencoder primarily removes information that isn’t perceptually important.

Then the diffusion model learns the higher-level distribution of the latent representation.

So:

Autoencoder → perceptual compression

 (111)

while:

Diffusion → semantic/generative modeling

 (112)

It’s not a strict separation in an information-theoretic sense, but it’s the useful conceptual picture.

31 Why This Improves High-Resolution Generation

Suppose you want a 1024×1024 image.

Pixel space contains:

$$1024^2 \times 3 = 3,145,728 \quad (113)$$

values.

With $f = 8$:

$$128 \times 128 \times 4 = 65,536 \quad (114)$$

That’s a massive reduction.

Therefore, the expensive iterative diffusion process operates on 65,536 latent values rather than 3,145,728 pixel values. Then one decoder transforms the final latent into the full-resolution image.

32 Why Can It Generate Megapixel Images Convolutionally?

This is one of the paper’s contributions.

Because the UNet is convolutional, it can operate on spatial latent maps.

Suppose:

$$z \in \mathbb{R}^{128 \times 128 \times c} \quad (115)$$

Convolution doesn’t fundamentally require a fixed global image size.

Therefore, for sufficiently suitable architectures, you can process larger spatial latent maps.

This allows synthesis of roughly 1024^2 images and beyond.

33 Why Not Use a Transformer Directly on Pixels?

Because image resolution creates huge token counts.

Suppose every pixel becomes a token. For 1024×1024 you have over one million spatial positions.

Self-attention has approximately:

$$O(N^2) \quad (116)$$

complexity with respect to token count N .

That’s disastrous at full pixel resolution.

LDM instead works with a spatially compressed representation. So the architecture is much more computationally manageable.

34 The Role of Convolution

The paper strongly benefits from the fact that its diffusion UNet is convolutional. Convolutions exploit:

- locality
- translation structure
- spatial hierarchy
- shared weights

This gives the diffusion model a natural inductive bias for images. The latent retains spatial structure, so convolution remains effective. This is why the authors can avoid extreme compression.

35 Full Mathematical Pipeline - Stages 1-6

Let's put everything together.

35.1 Stage 1: Autoencoder

$$z_0 = E(x_0) \tag{117}$$

and:

$$\hat{x} = D(z_0) \tag{118}$$

Train E, D so $\hat{x} \approx x_0$ perceptually.

35.2 Stage 2: Forward latent diffusion

$$q(z_t|z_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}z_0, (1 - \bar{\alpha}_t)I) \tag{119}$$

Equivalently:

$$z_t = \sqrt{\bar{\alpha}_t}z_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \tag{120}$$

35.3 Stage 3: Train latent denoiser

$$\epsilon_\theta(z_t, t) \tag{121}$$

Minimize:

$$\mathcal{L} = \mathbb{E} [\|\epsilon - \epsilon_\theta(z_t, t)\|^2] \tag{122}$$

35.4 Stage 4: Conditional version

Text:

$$y \rightarrow \tau_\theta(y) \quad (123)$$

Then:

$$\epsilon_\theta(z_t, t, \tau_\theta(y)) \quad (124)$$

35.5 Stage 5: Reverse diffusion

Start:

$$z_T \sim N(0, I) \quad (125)$$

Then:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_0 \quad (126)$$

35.6 Stage 6: Decode

$$\boxed{x = D(z_0)} \quad (127)$$

Done.

36 One Diagram to Memorize

You should memorize this:

TRAINING

RGB Image x

Encoder E

Latent z

Add noise

Noisy z timestep t

UNet

Predicted noise

MSE with

GENERATION

Gaussian noise

z_T

UNet text condition

z_{T-1}

repeat

z

Decoder D

Final Image

37 What is Actually Learned in Each Stage?

This is an excellent way to understand the paper.

Table 1: What Each Component Learns

Component	What it learns
Encoder E	Compact representation of images
Decoder D	Reconstruction from latent
Latent diffusion UNet	Distribution of realistic latent representations
ϵ_θ	
Text encoder τ_θ	Representation of conditioning information
Cross-attention	How conditioning information influences visual features

So:

$$\boxed{E, D \rightarrow \text{representation learning}} \quad (128)$$

while:

$$\boxed{\epsilon_\theta \rightarrow \text{generative modeling}} \quad (129)$$

and:

$$\boxed{\tau + \text{cross-attention} \rightarrow \text{conditional control}} \quad (130)$$

38 What the Latent is NOT

This is important.

The latent isn't necessarily:

- "This number means dog."
- "This channel means color."

The representation is distributed. Information is encoded across many latent dimensions and spatial locations.

So don't imagine:

$$z_1 = \text{dog} \quad (131)$$

$$z_2 = \text{color} \quad (132)$$

Instead, think:

$$z = \text{distributed representation of visual information} \quad (133)$$

39 Why the Decoder is So Important

Imagine two cases.

39.1 Case A

Excellent autoencoder:

$$x \rightarrow z \rightarrow \hat{x} \quad (134)$$

with:

$$\hat{x} \approx x \quad (135)$$

Then diffusion has a high-quality representation to model.

39.2 Case B

Poor autoencoder:

$$x \rightarrow z \rightarrow \hat{x} \quad (136)$$

where $\hat{x}$ is blurry and distorted.

Even if diffusion perfectly models z , the output remains limited by $D(z)$.

Therefore:

better latent representation $\Rightarrow$ higher possible generation quality

(137)

40 Why "Less Aggressive Downsampling" Matters

Suppose $f = 16$. Then:

$$512 \times 512 \rightarrow 32 \times 32 \quad (138)$$

This is computationally attractive. But a lot of spatial detail must be encoded into a tiny representation.

At $f = 4$, we have:

$$512 \times 512 \rightarrow 128 \times 128 \quad (139)$$

More information is preserved, but diffusion becomes more expensive.

So the paper searches for the middle ground. That's what Figure 1 demonstrates.

The objective isn't: Maximum compression. It is: Maximum useful compression without sacrificing perceptual quality.

41 Why the Authors Say "Near-Optimal Point"

There are two competing objectives:

41.1 Computational complexity

Want:

$$z \text{ small} \quad (140)$$

41.2 Reconstruction quality

Want:

$$z \text{ rich} \tag{141}$$

Therefore:

optimal latent size = trade-off between efficiency and fidelity

(142)

This is the "near-optimal point between complexity reduction and detail preservation" mentioned in the abstract.

42 Why Diffusion is Particularly Suited to the Latent Space

This is another subtle but important argument.

The latent is still:

$$h \times w \times c \tag{143}$$

So it maintains spatial structure. Therefore a UNet can still perform:

- local convolution
- multi-scale feature extraction

If you destroyed spatial structure entirely and represented the image as an arbitrary vector, you would lose some of the natural image inductive bias.

LDM preserves enough spatial organization to retain the benefits of convolution.

43 Why This is Better Than the Earlier VQGAN + Transformer Idea

Earlier approaches often did:

$$x \rightarrow \text{highly compressed discrete latent} \rightarrow \text{Transformer} \rightarrow \text{latent} \rightarrow x \tag{144}$$

The latent might be represented as a sequence:

$$[z_1, z_2, \dots, z_N] \tag{145}$$

The Transformer then models the sequence.

The problem is that high-resolution images require many tokens.

LDM says: We don't need to flatten away the spatial structure. Let's use a convolutional diffusion model directly on the spatial latent.

Thus:

spatial latent + UNet

(146)

is much more scalable.

44 How Image-to-Image Tasks Fit Into This

The framework isn't limited to text-to-image.

Because the diffusion model operates in latent space, you can condition generation on other things.

For example:

44.1 Inpainting

Given an image with a missing region:

$$x_{\text{masked}} \tag{147}$$

condition the diffusion process to fill the missing area.

44.2 Super-resolution

Given low-resolution image:

$$x_{\text{low-res}} \tag{148}$$

generate a high-resolution latent conditioned on it.

44.3 Semantic synthesis

Given a semantic map:

$$y_{\text{semantic}} \tag{149}$$

generate an image consistent with that map.

44.4 Text-to-image

Given:

$$y_{\text{text}} \tag{150}$$

generate an image matching the text.

So the framework is:

$$\boxed{p(z|y)} \tag{151}$$

where y can represent different modalities.

45 Why Cross-Attention is Called "General-Purpose Conditioning"

Suppose y is:

45.1 Text

$$y = \text{text} \tag{152}$$

Use a text encoder.

45.2 Semantic map

$$y = \text{semantic representation} \tag{153}$$

Use a suitable encoder.

45.3 Bounding boxes

$$y = \text{object locations} \tag{154}$$

Use another encoder.

As long as we can produce $\tau_\theta(y)$, the UNet can potentially interact with it through cross-attention.

That’s why the paper describes this as a flexible conditioning mechanism.

46 One Subtle Point

Latent diffusion doesn’t eliminate the autoencoder cost.

It reduces the cost of the iterative diffusion computation.

You still have E and D . But during training, the encoder can be used to precompute latents, and during inference the decoder is used only once.

So the expensive repeated computation:

$$T \times \text{UNet} \tag{155}$$

happens in low-dimensional space.

That is where the huge gain comes from.

47 Training Versus Inference

This distinction is useful.

47.1 Training

For each image:

$$x \rightarrow E(x) \rightarrow z_0 \rightarrow z_t \rightarrow \epsilon_\theta \tag{156}$$

The diffusion model is trained in latent space.

47.2 Inference

No input image is required for text-to-image.

Start:

$$z_T \sim N(0, I) \quad (157)$$

Then:

$$T \text{ denoising steps} \quad (158)$$

and finally:

$$D(z_0) \quad (159)$$

48 Why This Was Such an Important Paper

Before this work, diffusion models were powerful but computationally expensive because they operated directly on pixels.

LDM introduced a practical decomposition:

$$\boxed{\text{perceptual compression} + \text{latent diffusion}} \quad (160)$$

This made high-resolution diffusion much more accessible.

Then adding:

$$\boxed{\text{cross-attention}} \quad (161)$$

made conditional generation extremely flexible.

Together, these ideas became the conceptual foundation of modern latent text-to-image diffusion systems.

49 The Entire Paper in One Mathematical Chain

If you remember only one thing, remember this:

$$\boxed{x \xrightarrow{E} z_0 \xrightarrow{q} z_t \xrightarrow{\epsilon_\theta(z_t, t, y)} z_{t-1} \rightarrow \dots \rightarrow z'_0 \xrightarrow{D} x'} \quad (162)$$

For unconditional generation:

$$\epsilon_\theta(z_t, t) \quad (163)$$

For conditional generation:

$$\epsilon_\theta(z_t, t, \tau(y)) \quad (164)$$

And cross-attention connects:

$$\boxed{\text{UNet features} \leftrightarrow \text{conditioning embeddings}} \quad (165)$$

50 The Deepest Intuition

Imagine you're asked to recreate a photograph.

A naïve diffusion model says: "I'll learn how every single pixel behaves."

LDM says: "First, let's transform the photograph into a compact representation that preserves what matters visually. Then I'll learn the probability distribution of those representations. Finally, I'll turn the generated representation back into pixels."

So:

$$\boxed{\text{pixels are the output space, not necessarily the best generation space.}} \quad (166)$$

That's the philosophical core of the paper.

51 Final Mental Model

Think of LDM as three brains working together:

51.1 Brain 1 — Encoder

$$x \rightarrow z \quad (167)$$

"Compress the image intelligently."

51.2 Brain 2 — Diffusion UNet

$$z_T \rightarrow z_0 \quad (168)$$

"Generate a realistic latent representation by gradually removing noise."

51.3 Brain 3 — Decoder

$$z \rightarrow x \quad (169)$$

"Turn the latent representation back into a high-resolution image."

And if text is involved:

51.4 Controller — Text encoder + cross-attention

$$y \rightarrow \tau(y) \quad (170)$$

"Tell the diffusion UNet what kind of latent it should generate."

Therefore:

$$\boxed{\text{Image} \leftrightarrow \text{Latent Representation} \leftrightarrow \text{Diffusion}} \quad (171)$$

with:

$$\boxed{\text{Text} \rightarrow \text{Cross-Attention} \rightarrow \text{Diffusion}} \quad (172)$$

That is the essence of Latent Diffusion Models.

52 Why L2 Creates Blur (The Autoencoder Problem)

Suppose:

$$x = \text{original image} \quad (173)$$

and the autoencoder reconstructs:

$$\tilde{x} = \mathcal{D}(\mathcal{E}(x)) \quad (174)$$

A simple reconstruction objective is:

$$\mathcal{L}_{L2} = \|x - \tilde{x}\|_2^2 \quad (175)$$

Why does L2 create blur?

Consider a very simple 1D example. Suppose the training data contains two equally plausible possibilities for an edge:

$$x_A = [0, 0, 0, 1, 1, 1] \quad (176)$$

or:

$$x_B = [0, 0, 1, 1, 1, 1] \quad (177)$$

The edge is shifted by one pixel.

If the model cannot determine which position is correct, minimizing the expected squared error encourages it toward the conditional mean:

$$\tilde{x} \approx \frac{x_A + x_B}{2} \quad (178)$$

which gives:

$$\tilde{x} \approx [0, 0, 0.5, 1, 1, 1] \quad (179)$$

Instead of a sharp edge $[0, 0, 0, 1, 1, 1]$, we get $[0, 0, 0.5, 1, 1, 1]$.

That intermediate value represents a soft/blurred edge.

This is the fundamental problem:

L2 favors averaging plausible solutions

(180)

rather than selecting one visually plausible solution.

53 What the LDM Paper Does Instead

Rombach et al. use a perceptual loss together with an adversarial objective for the autoencoder.

Conceptually:

$\mathcal{L}_{\text{AE}} = \mathcal{L}_{\text{perceptual}} + \lambda \mathcal{L}_{\text{GAN}} + \text{reconstruction-related terms}$

(181)

The perceptual component asks something closer to: "Does the reconstructed image look like the original at a semantic/visual-feature level?" rather than simply: "Is every pixel numerically close?"

And the adversarial component encourages the reconstruction to lie on the distribution of realistic images.

So instead of forcing:

$$\tilde{x} \approx x \quad (182)$$

purely in pixel space, the autoencoder is encouraged to produce:

$$\boxed{\tilde{x} \approx x \text{ perceptually}} \quad (183)$$

while maintaining realistic image statistics.

54 Model B — The Diffusion Model Takes Over

If the autoencoder gives us:

$$x \xrightarrow{\mathcal{E}} z \quad (184)$$

then the diffusion model asks: which regions of latent space correspond to realistic images, and how can I sample from those regions?

54.1 What It Learns

During training, we take a real latent z_0 and gradually add Gaussian noise:

$$z_0 \rightarrow z_1 \rightarrow z_2 \rightarrow \cdots \rightarrow z_T \quad (185)$$

The forward process eventually turns the meaningful latent into approximately Gaussian noise:

$$z_T \sim \mathcal{N}(0, I) \quad (186)$$

The diffusion model then learns the reverse process:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_0 \quad (187)$$

So, in simple terms:

$$\boxed{\text{noise} \rightarrow \text{realistic latent}} \quad (188)$$

and finally the decoder converts that latent back to an image:

$$\boxed{z_0 \rightarrow \mathcal{D}(z_0) = x} \quad (189)$$

54.2 What Does the Neural Network Actually Learn?

At each timestep t , the network receives a noisy latent z_t and learns to predict the noise that was added:

$$\epsilon_\theta(z_t, t) \quad (190)$$

The common training objective is:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{z_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(z_t, t)\|_2^2] \quad (191)$$

Therefore, it isn't directly learning: "This latent is realistic." Rather, it learns a denoising function that gradually moves noisy latent representations toward the manifold/distribution of realistic latents.

55 The Whole LDM Pipeline

Think of the two models as having very different jobs:

$$\boxed{\underbrace{x \rightarrow z}_{\text{Model A: Autoencoder}}} \quad (192)$$

Model A: "How can I represent an image compactly while preserving perceptually important information?"

Then:

$$\boxed{\underbrace{z_T \rightarrow z_0}_{\text{Model B: Diffusion}}} \quad (193)$$

Model B: "How do I transform random noise into a latent that looks like it came from the distribution of real images?"

Finally:

$$\boxed{z_0 \rightarrow \hat{x}} \quad (194)$$

The decoder reconstructs the image.

So the key separation is:

$$\boxed{\text{Autoencoder} = \text{learn the representation}} \quad (195)$$

$$\boxed{\text{Diffusion} = \text{learn the distribution of that representation}} \quad (196)$$

This is why LDMs can perform diffusion in latent space rather than expensive pixel space.

56 But "Accurately" Has Two Meanings

The diffusion model does not try to reproduce one particular training latent. It learns the distribution $p_{\text{data}}(z)$ and generates new samples from it.

For example, if the training images contain many different faces:

$$z_1, z_2, z_3, \dots, z_N \quad (197)$$

the diffusion model learns the structure of the space containing those face representations. It can then generate a new z that wasn't necessarily present in the training set.

So:

$$\boxed{\text{learn latent distribution} \Rightarrow \text{sample plausible new latents} \Rightarrow \text{decode into plausible new images}} \quad (198)$$

That's why saying "diffusion learns how to generate realistic latent representations" is a very good mental model.

57 Training is Conceptually Done in Two Stages

57.1 Stage 1 — Train the Autoencoder

First train:

$$\boxed{x \xrightarrow{E} z \xrightarrow{D} \tilde{x}} \quad (199)$$

where:

- E = encoder
- z = latent feature map
- D = decoder
- $\tilde{x}$ = reconstructed image

The goal is to make $\tilde{x} \approx x$ using the perceptual/reconstruction/adversarial objectives we discussed.

After training, the encoder and decoder provide a useful latent space.

57.2 Stage 2 — Train the Diffusion Model

Now freeze/use the trained encoder and convert training images into latents:

$$x \rightarrow E(x) = z_0 \quad (200)$$

Then add noise:

$$z_0 \rightarrow z_t \quad (201)$$

and train the diffusion network to denoise it:

$$z_t \rightarrow z_0 \quad (202)$$

So the diffusion model learns the distribution of these learned latent representations.

57.3 During Generation

You don't start with an image. You start with random noise:

$$z_T \sim \mathcal{N}(0, I) \quad (203)$$

Then diffusion gradually generates a realistic latent:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_0 \quad (204)$$

Finally, use the already-trained decoder:

$$z_0 \xrightarrow{D} \hat{x} \quad (205)$$

So the complete picture is:

$$\boxed{\text{Stage 1: } x \xrightarrow{E} z \xrightarrow{D} \tilde{x}} \quad (206)$$

then:

$$\boxed{\text{Stage 2: } z_0 \rightarrow z_t \quad \text{and learn} \quad z_t \rightarrow z_0} \quad (207)$$

and at inference:

$$\boxed{\text{noise} \rightarrow \text{diffusion} \rightarrow \text{latent} \rightarrow \text{decoder} \rightarrow \text{image}} \quad (208)$$

So yes: first learn how to represent/reconstruct images, then learn how to generate those representations.

58 Complete Training Map

STAGE 1
AUTOENCODER TRAINING

Training image x

Encoder
E

Latent z

Decoder
D

Reconstructed $\tilde{x}$

Reconstruction /
Perceptual / GAN losses

Update E and D

After Stage 1: E, D are trained.

STAGE 2
DIFFUSION TRAINING

Training image x

Trained Encoder E

z

add noise

z

Diffusion
Model

predicted noise

Diffusion loss

Update diffusion model

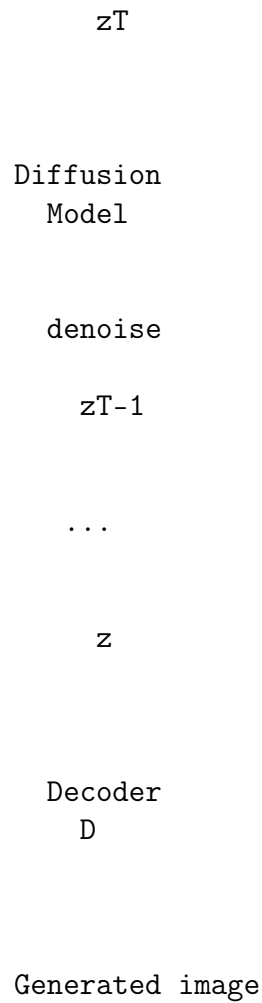
During Stage 2:

$$\boxed{x \rightarrow E(x) \rightarrow z_0} \tag{209}$$

Then diffusion learns from z_0 .

59 Complete Generation Map

Random Gaussian noise



60 The Simplest Mental Model

Think of the two stages as two different questions.

60.1 Stage 1 — Autoencoder

"How do I represent an image compactly?"

(210)

It learns:

$$x \leftrightarrow z \tag{211}$$

60.2 Stage 2 — Diffusion

"What does a realistic image representation look like, and how can I generate one from noise?"

(212)

It learns:

$$\text{noise} \rightarrow z \tag{213}$$

60.3 Final System

Therefore:

$$\boxed{\underbrace{x \rightarrow z}_{\text{Encoder}} \quad \underbrace{\text{noise} \rightarrow z}_{\text{Diffusion}} \quad \underbrace{z \rightarrow x}_{\text{Decoder}}} \quad (214)$$

Or, for generation specifically:

$$\boxed{\mathcal{N}(0, I) \xrightarrow{\text{Diffusion}} z_0 \xrightarrow{\text{Decoder}} \text{new image}} \quad (215)$$

That is the central mechanism of Latent Diffusion Models: the autoencoder makes the representation manageable; diffusion learns the distribution of that representation; the decoder turns generated latents back into images.

61 Text Conditioning During Training AND Generation

Text conditioning happens during both training and generation, but it plays different roles.

For a text-to-image LDM such as Stable Diffusion:

61.1 During Training

You have an image x and its text prompt c .

First:

$$x \xrightarrow{E} z_0 \quad (216)$$

Then add noise:

$$z_0 \rightarrow z_t \quad (217)$$

The text is converted into an embedding:

$$\text{text} \xrightarrow{\text{text encoder}} c \quad (218)$$

The diffusion model receives both:

$$\boxed{(z_t, t, c)} \quad (219)$$

and learns to predict the noise:

$$\epsilon_\theta(z_t, t, c) \quad (220)$$

So training teaches the model: "Given this noisy latent and this text description, how should I denoise the latent?"

61.2 During Generation

You provide a new text prompt: "A dog running on a beach"

The text encoder produces:

$$c = \text{TextEncoder}(\text{prompt}) \quad (221)$$

Then start with random noise:

$$z_T \sim \mathcal{N}(0, I) \quad (222)$$

and repeatedly denoise while providing the text condition:

$$(z_T, c) \rightarrow (z_{T-1}, c) \rightarrow \cdots \rightarrow (z_0, c) \quad (223)$$

Finally:

$$z_0 \xrightarrow{D} \text{image} \quad (224)$$

So:

Text conditioning is learned during training AND supplied during generation.

 (225)

61.3 One Important Distinction

The text encoder may be pretrained and kept frozen, depending on the LDM implementation.

TRAINING

Image Autoencoder z add noise z

Text Text Encoder c

Diffusion U-Net

predicted noise

Then during generation:

Text Text Encoder c

Random noise Diffusion

generated z

Decoder

Image

So the key point is: training teaches the diffusion model how to use text to guide denoising; generation actually uses that learned text-conditioning mechanism to control what image is produced.

62 Semantic Mapping vs Semantic Synthesis

62.1 What is a Semantic Map?

A semantic map is an image-like representation where each pixel/region indicates what object or class is present there, rather than its RGB appearance.

For example, consider a street scene:

```

      SKY SKY SKY SKY SKY
    SKY SKY SKY SKY SKY
BUILDING BUILDING  TREE
BUILDING   CAR    TREE
ROAD      ROAD    ROAD
ROAD      ROAD    ROAD

```

A real semantic map might assign labels such as:

$$\text{sky} = 0, \quad \text{road} = 1, \quad \text{car} = 2, \quad \text{building} = 3, \quad \text{tree} = 4 \quad (226)$$

So instead of saying "This pixel is RGB = (125, 170, 210)", the semantic map says "This pixel belongs to the sky."

62.2 Semantic Mapping

Semantic mapping means converting an image or scene into a representation describing its semantic content.

Conceptually:

$$\boxed{\text{RGB Image} \rightarrow \text{Semantic Map}} \quad (227)$$

For example:

Original image

Semantic segmentation model

```

Sky
Sky      Building
Tree     Car
Road     Road

```

The semantic map contains information like: where the road is, where the cars are, where buildings are, where people are, where trees are, etc.

It focuses on meaning, not appearance.

62.3 Semantic Synthesis

Semantic synthesis goes in the opposite direction.

You provide the semantic description:

semantic map (228)

and the model generates a realistic image:

Semantic Map $\rightarrow$ Realistic Image (229)

For example:

Semantic map

```
SKY SKY SKY SKY
BUILDING TREE
BUILDING CAR TREE
ROAD ROAD ROAD
```

LDM / Diffusion

Realistic image

The generated image might contain:

- realistic blue sky
- realistic building textures
- realistic tree leaves
- realistic car appearance
- realistic road texture

even though the input only told the model what is where.

62.4 The Difference in One Line

Table 2: Semantic Mapping vs Semantic Synthesis

Concept	Direction	Purpose
Semantic mapping	Image $\rightarrow$ semantic representation	Understand the scene
Semantic synthesis	Semantic representation $\rightarrow$ image	Generate the scene

So:

Semantic Mapping: $x \rightarrow y$ (230)

where y is semantic information.
And:

$$\boxed{\text{Semantic Synthesis: } y \rightarrow x} \quad (231)$$

62.5 Why is This Relevant to the LDM Paper?

The paper says that LDMs can handle semantic synthesis.

Suppose y is a semantic map. The conditional diffusion model learns:

$$p(z|y) \quad (232)$$

Remember that diffusion operates on the latent z .

So the process becomes:

$$\boxed{\text{Semantic Map} \rightarrow \text{Conditioning Encoder} \rightarrow \text{Cross-Attention} \rightarrow \text{Latent Diffusion} \rightarrow \text{Decoder} \rightarrow \text{Realistic Image}} \quad (233)$$

62.6 What Happens Internally?

Suppose the semantic map says:

SKY SKY SKY
BUILDING BUILDING
BUILDING TREE
ROAD ROAD
CAR

The conditioning encoder transforms this into some representation:

$$\tau(y) \quad (234)$$

Then the diffusion UNet receives:

$$\epsilon_{\theta}(z_t, t, \tau(y)) \quad (235)$$

At cross-attention layers:

$$Q = W_Q \phi(z_t) \quad (236)$$

comes from the noisy image latent, while:

$$K = W_K \tau(y) \quad (237)$$

and:

$$V = W_V \tau(y) \quad (238)$$

come from the semantic condition.

Thus the UNet can use the semantic information while denoising.

62.7 Why Does the Semantic Map Not Specify the Exact Pixels?

This is the beautiful part.

Suppose the semantic map says CAR at a particular location. It doesn't specify:

- exact car color
- exact texture
- reflections
- wheel appearance
- lighting
- shadows
- small details

The diffusion model has learned these from the training data.

Therefore, many different realistic images can correspond to the same semantic map.

For example, $y = \text{"car on road"}$ could generate:

- Image 1: red car
- Image 2: blue car
- Image 3: black car

All satisfy the same semantic structure.

This is why the model is generative rather than simply performing deterministic rendering.

62.8 Semantic Synthesis is Therefore a Conditional Distribution

Mathematically, you're not learning just $x = f(y)$. You're learning:

$$\boxed{p(x|y)} \tag{239}$$

or, because LDM works in latent space:

$$\boxed{p(z|y)} \tag{240}$$

Then:

$$z \sim p(z|y) \tag{241}$$

and:

$$x = D(z) \tag{242}$$

This allows multiple valid images for the same semantic map.

62.9 Example: Semantic Synthesis

Suppose your semantic map is:

```
SKY
  SKY
BUILDING  TREE
BUILDING CAR  TREE
ROAD ROAD ROAD
```

The model understands approximately: there should be a building here, a tree here, a car here, and a road underneath.

But it decides the actual visual realization:

```
blue sky

realistic building  green tree
realistic building  red car
gray road
```

The semantic map provides structure. The diffusion model provides appearance. That's the key idea:

$$\boxed{\text{Semantic condition} = \text{WHAT and WHERE}} \quad (243)$$

while:

$$\boxed{\text{Diffusion} = \text{HOW IT LOOKS}} \quad (244)$$

62.10 Semantic Mapping vs Semantic Synthesis in the LDM Context

You can visualize the complete cycle as:

```
UNDERSTANDING

RGB IMAGE

Semantic Mapping

SEMANTIC MAP

Semantic Synthesis

GENERATED IMAGE
```

But LDM mainly focuses on the generation direction:

$$\boxed{\text{semantic map} \rightarrow \text{image}} \quad (245)$$

rather than being primarily a semantic segmentation model.

62.11 The Easiest Way to Remember It

Semantic mapping: "Tell me what is present and where."

$$\boxed{\text{Image} \rightarrow \text{Meaning}} \quad (246)$$

Semantic synthesis: "Given what should be present and where, create a realistic image."

$$\boxed{\text{Meaning} \rightarrow \text{Image}} \quad (247)$$

And in LDM:

$$\boxed{\text{Semantic Map} \rightarrow \text{Cross-Attention} \rightarrow \text{Latent Diffusion} \rightarrow \text{Decoder} \rightarrow \text{Realistic Image}} \quad (248)$$

63 Applications of Latent Diffusion Models

These are all parts of the LDM paper's application/architecture discussion. The unifying idea is that once you have a good latent space, you don't have to use diffusion only for unconditional image generation—you can use the latent diffusion model for inpainting, super-resolution, conditional generation, text conditioning, and high-resolution synthesis.

63.1 Inpainting with Latent Diffusion

63.1.1 What is Inpainting?

Suppose you have an image with a missing region:

Original image:

person

← missing region

The goal is:

$$\boxed{\text{fill the missing region realistically}} \quad (249)$$

while preserving the known part of the image.

63.1.2 How LDM Does This

Let x be the original image and M be a binary mask.

For example:

$$M_{ij} = \begin{cases} 1 & \text{missing} \\ 0 & \text{known} \end{cases} \quad (250)$$

You have x_{known} and want to generate x_{missing} .

63.1.3 The Important Idea

Instead of asking diffusion to generate an entire image from scratch, we condition it on the known part of the image.

Conceptually:

$$\boxed{p(z_{\text{missing}} \mid z_{\text{known}})} \quad (251)$$

The model generates only the uncertain content while respecting the information that is already known.

63.1.4 Latent-Space Version

First encode the image:

$$x \rightarrow z \quad (252)$$

The mask is also represented appropriately in the latent space.

The diffusion model receives information about:

$$\boxed{\text{noisy latent} + \text{mask} + \text{known image content}} \quad (253)$$

and predicts the missing content.

During sampling, the known region is constrained while the unknown region is progressively generated.

Original image

Encoder

latent representation

known region

Conditioning

Random noise Diffusion

completed latent

Decoder

Completed image

63.1.5 Why Latent Diffusion Helps

Pixel-space inpainting would run diffusion over the full high-resolution image. LDM instead operates on $z \ll x$, so the denoising process is substantially cheaper.

The LDM paper reports state-of-the-art inpainting results for its time while requiring significantly less computation than pixel-space approaches.

63.2 Super-Resolution with Latent Diffusion

Super-resolution means:

$$\boxed{\text{low-resolution image} \rightarrow \text{high-resolution image}} \quad (254)$$

For example:

$$128 \times 128 \rightarrow 512 \times 512 \quad (255)$$

The problem is that a low-resolution image does not uniquely determine its high-resolution version.

For example:

Low-resolution:
blurry face

Possible HR images:

face A face B face C

The diffusion model can learn to choose a plausible high-resolution realization.

63.2.1 Conditional Diffusion

Instead of $p(z)$, we learn:

$$\boxed{p(z_{\text{HR}} \mid x_{\text{LR}})} \quad (256)$$

The low-resolution image becomes the condition.

The process is:

$$x_{LR} \rightarrow \text{conditioning representation} \quad (257)$$

while diffusion starts from noise:

$$z_T \sim \mathcal{N}(0, I) \quad (258)$$

and generates:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_0 \quad (259)$$

conditioned on the low-resolution image.

Finally:

$$z_0 \rightarrow D(z_0) \quad (260)$$

gives the high-resolution image.

63.2.2 Why This is Different from Simply Upscaling

Normal interpolation:

$$128^2 \rightarrow 512^2 \quad (261)$$

only estimates missing pixels mathematically.

Diffusion asks: What high-resolution image is plausible given this low-resolution evidence?

Therefore it can generate realistic:

- textures
- edges
- fine structures
- details

that were not explicitly present in the low-resolution input.

This is fundamentally a conditional generation problem.

63.3 Conditional Latent Diffusion

This is one of the most important ideas in the paper.

Instead of learning $p(z)$, we learn:

$$\boxed{p(z \mid y)} \quad (262)$$

where y is some condition.

The condition could be:

- text
- class label
- semantic map

- segmentation
- bounding box
- low-resolution image
- another image
- layout information

The LDM paper specifically emphasizes that its cross-attention mechanism allows diffusion models to accept general conditioning inputs such as text and bounding boxes.

63.3.1 How Conditioning Enters the Model

Suppose z_t is the noisy latent. The condition is y .

The diffusion model becomes:

$$\boxed{\epsilon_{\theta}(z_t, t, y)} \quad (263)$$

instead of:

$$\epsilon_{\theta}(z_t, t) \quad (264)$$

So training becomes:

$$\mathcal{L} = \mathbb{E} [\|\epsilon - \epsilon_{\theta}(z_t, t, y)\|^2] \quad (265)$$

63.4 Transformer Encoders for LDM

This section is about the conditioning encoder, particularly when the condition is something like text.

Suppose the prompt is: "A dog running on a beach"

A text encoder converts this sequence into feature vectors:

$$y_1, y_2, \dots, y_L \quad (266)$$

A Transformer encoder is useful because it can understand relationships between tokens.

For example:

$$\text{dog} \leftrightarrow \text{running} \leftrightarrow \text{beach} \quad (267)$$

rather than treating each word independently.

The output becomes:

$$\boxed{c \in \mathbb{R}^{L \times d}} \quad (268)$$

This c is then fed into the diffusion model through cross-attention.

So:

Text

Tokenizer

tokens

Transformer encoder

text embeddings

latent cross-attention

Diffusion U-Net

generated latent

The key point is that the Transformer is not necessarily the diffusion model itself in the original LDM architecture. It can be the conditioning encoder.

This distinction is important.

63.5 Convolutional Sampling Beyond 256^2

This is a particularly interesting part of LDM.

Suppose you want 1024×1024 images.

A normal Transformer operating on every pixel would have an enormous number of tokens.

LDM instead performs diffusion on a spatial latent representation.

For example:

$$1024 \times 1024 \rightarrow 128 \times 128 \times C \tag{269}$$

if the autoencoder uses an 8x spatial compression.

The diffusion model therefore operates on 128×128 rather than 1024×1024 .

63.5.1 Why Convolution Helps

Convolutions are naturally spatial. A convolution can process $H \times W \times C$ without converting every pixel into a huge sequence of tokens.

Therefore the model can operate fully convolutionally.

This means the same architecture can be applied to different spatial sizes.

For example:

- 64×64
- 128×128
- 256×256

latent feature maps.

This allows the model to synthesize images beyond 256^2 without requiring a massive pixel-space representation. The paper explicitly highlights high-resolution synthesis through a convolutional formulation.

63.6 Image Generation with Latent Diffusion

This is the simplest case. There is no external condition.

You learn $p(z)$ rather than $p(z|y)$.

63.6.1 Training

Take an image x .

Encode:

$$z_0 = E(x) \tag{270}$$

Add noise:

$$z_t = \alpha_t z_0 + \sigma_t \epsilon \tag{271}$$

Train:

$$\epsilon_\theta(z_t, t) \tag{272}$$

to predict ϵ using:

$$\mathcal{L} = \|\epsilon - \epsilon_\theta(z_t, t)\|^2 \tag{273}$$

63.6.2 Generation

Start:

$$z_T \sim \mathcal{N}(0, I) \tag{274}$$

Then:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_0 \tag{275}$$

Finally:

$$\boxed{z_0 \xrightarrow{D} \hat{x}} \tag{276}$$

So the model generates a completely new image.

63.7 On Perceptual Compression Tradeoffs

This is one of the most important sections for understanding why LDM works.

The autoencoder has a compression factor.

Suppose $512 \times 512 \times 3$ is compressed to $64 \times 64 \times 4$. That's a large reduction in dimensionality.

But there is a tradeoff.

63.7.1 Too Little Compression

Suppose:

$$512 \times 512 \times 3 \rightarrow 256 \times 256 \times C \quad (277)$$

The latent is still very large. Therefore:

less compression $\Rightarrow$ more computation

 (278)

But you preserve lots of detail.

63.7.2 Too Much Compression

Suppose:

$$512 \times 512 \times 3 \rightarrow 16 \times 16 \times C \quad (279)$$

Now the latent is tiny. Diffusion becomes extremely cheap.

But the autoencoder may lose:

- texture
- edges
- fine details
- small objects
- spatial information

So:

more compression $\Rightarrow$ less computation but more information loss

 (280)

63.7.3 The Sweet Spot

This is exactly where LDM's main contribution comes in.

They want:

enough compression to make diffusion cheap

 (281)

but:

not so much compression that visual information disappears

 (282)

So:

$$\boxed{\text{moderate compression} + \text{perceptually good reconstruction}} \quad (283)$$

creates a good latent space for diffusion.

The paper describes this as finding a near-optimal point between complexity reduction and detail preservation.

63.8 Why Pixel-Wise MSE Matters Here

Suppose the autoencoder uses only:

$$\|x - \tilde{x}\|_2^2 \quad (284)$$

It may preserve low-level pixel accuracy but produce blurry reconstructions.

The paper therefore uses perceptual objectives so that the autoencoder prioritizes perceptually important information.

That means the latent doesn't have to preserve every pixel equally. Instead, it should preserve information that matters for:

$$\boxed{\text{human-perceived visual quality}} \quad (285)$$

This is extremely important.

The autoencoder is effectively saying: "I don't need to remember every pixel exactly. I need to remember enough information to reconstruct an image that looks correct."

That is what makes stronger compression possible without catastrophic perceptual degradation.

64 The Complete Connection Between All These Sections

Now you can see how all the sections fit together:

ORIGINAL IMAGE

Encoder

LATENT FEATURE MAP

Image Generation Inpainting Super-resolution

LATENT DIFFUSION

unconditional

conditional

Text
condition

Image/Mask/
Layout/etc.

Transformer/
Text Encoder

Cross-Attention

GENERATED LATENT

Decoder

GENERATED IMAGE

And the perceptual compression tradeoff sits underneath the entire architecture:

$$\boxed{\text{Image} \xrightarrow{\text{perceptual compression}} \text{compact latent} \xrightarrow{\text{diffusion}} \text{new latent} \xrightarrow{\text{decoder}} \text{image}} \quad (286)$$

65 One-Line Meaning of Each Section

Table 3: One-Line Meaning of Each Section

Section	Main Idea
Inpainting with LDM	Generate missing image regions while preserving known regions
Super-Resolution with LDM	Generate a plausible high-resolution image conditioned on low-resolution information
Conditional LDM	Learn $p(z y)$ instead of only $p(z)$
Transformer Encoders for LDM	Convert structured conditions such as text into useful embeddings
Convolutional Sampling Beyond 256^2	Exploit spatial convolutions in latent space to efficiently synthesize high-resolution images
Image Generation with LDM	Generate new images by sampling latent noise and denoising it
Perceptual Compression Tradeoffs	Find the balance between latent size/computational cost and visual information preservation

66 The Big Picture

The big picture is therefore:

$$\boxed{\underbrace{\text{Perceptual Autoencoder}}_{\text{compress intelligently}} + \underbrace{\text{Latent Diffusion}}_{\text{learn/generate latent distribution}} + \underbrace{\text{Conditioning}}_{\text{control generation}}} \quad (287)$$

That combination is what makes LDM substantially more computationally attractive while retaining high visual quality.

67 Mathematical Derivation of the Diffusion Objective

This section derives why the diffusion training objective becomes a simple noise-prediction MSE.

67.1 Forward Diffusion Starts from x_0

You have a clean data sample x_0 .

The forward process gradually destroys information by adding Gaussian noise. They define:

$$\boxed{\text{SNR}(t) = \frac{\alpha_t^2}{\sigma_t^2}} \quad (288)$$

where:

- α_t controls how much of the original signal remains

- σ_t controls how much noise is present

The noisy sample at time t is:

$$\boxed{x_t = \alpha_t x_0 + \sigma_t \epsilon} \quad (289)$$

where:

$$\epsilon \sim \mathcal{N}(0, I) \quad (290)$$

This is equation (4):

$$q(x_t|x_0) = \mathcal{N}(x_t|\alpha_t x_0, \sigma_t^2 I) \quad (291)$$

So instead of adding noise one step at a time during training, we can directly generate x_t from x_0 .

67.2 Markov Relationship Between Two Time Steps

For $s < t$:

$$q(x_t|x_s) = \mathcal{N}(x_t|\alpha_{t|s}x_s, \sigma_{t|s}^2 I) \quad (292)$$

with:

$$\alpha_{t|s} = \frac{\alpha_t}{\alpha_s} \quad (293)$$

and:

$$\sigma_{t|s}^2 = \sigma_t^2 - \alpha_{t|s}^2 \sigma_s^2 \quad (294)$$

This simply describes $x_s \rightarrow x_t$ when moving forward through the diffusion process.

67.3 Now Reverse the Process

The forward process is:

$$\boxed{x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T} \quad (295)$$

Eventually:

$$x_T \approx \mathcal{N}(0, I) \quad (296)$$

The generative model wants to reverse this:

$$\boxed{x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_1 \rightarrow x_0} \quad (297)$$

The reverse model is:

$$p_\theta(x_{t-1}|x_t) \quad (298)$$

Therefore generation becomes:

$$p(x_0) = \int p(x_T) \prod_{t=1}^T p(x_{t-1}|x_t) dx_{1:T} \quad (299)$$

That's equation (8).

67.4 Why Do We Need ELBO?

We want to maximize $\log p(x_0)$, but directly computing $p(x_0)$ is difficult.

So we use the Evidence Lower Bound (ELBO).

The negative log likelihood is bounded by:

$$-\log p(x_0) \leq \text{KL}(q(x_T|x_0)|p(x_T)) + \sum_t \mathbb{E}_q [\text{KL}(q(x_{t-1}|x_t, x_0)|p_\theta(x_{t-1}|x_t))] \quad (300)$$

This is equation (9).

The important part is:

$$q(x_{t-1}|x_t, x_0) \quad (301)$$

versus:

$$p_\theta(x_{t-1}|x_t) \quad (302)$$

We know the first one mathematically because the forward process is known. But the reverse process is what we need to learn.

67.5 The Key Problem: Reverse Process Needs x_0

The true reverse posterior is:

$$q(x_{t-1}|x_t, x_0) \quad (303)$$

Notice that it requires x_0 . But during generation, we only have x_t . We don't know the original x_0 .

So the model needs to estimate x_0 .

They introduce:

$$x_\theta(x_t, t) \quad (304)$$

which means: given the noisy sample x_t at timestep t , predict what the original clean sample x_0 was.

Then they define:

$$p_\theta(x_{t-1}|x_t) = q(x_{t-1}|x_t, x_\theta(x_t, t)) \quad (305)$$

This is equation (10).

That's a very important idea.

67.6 The Reverse Step Becomes Gaussian

The posterior can be written as:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1} \mid \mu_\theta(x_t, t), \sigma_{t|t-1}^2 \frac{\sigma_{t-1}^2}{\sigma_t^2} I\right) \quad (306)$$

The mean is:

$$\mu_\theta(x_t, t) = \frac{\alpha_{t|t-1}\sigma_{t-1}^2}{\sigma_t^2}x_t + \frac{\alpha_{t-1}\sigma_{t|t-1}^2}{\sigma_t^2}x_\theta(x_t, t) \quad (307)$$

The important conceptual point is:

$$\boxed{x_\theta(x_t, t) \text{ gives us an estimate of } x_0} \quad (308)$$

and that estimate is used to determine the mean of the previous denoising step.

67.7 ELBO Becomes an x_0 -Prediction Objective

After substituting this parameterization into the ELBO, the complicated KL terms simplify to:

$$\sum_t \mathbb{E} \left[\frac{1}{2} (\text{SNR}(t-1) - \text{SNR}(t)) \|x_0 - x_\theta(x_t, t)\|^2 \right] \quad (309)$$

So the model is essentially being asked:

$$\boxed{\text{Given } x_t, \text{ predict } x_0} \quad (310)$$

That's equation (13).

But there's an even more convenient parameterization.

67.8 Instead of Predicting x_0 , Predict the Noise

Remember:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon \quad (311)$$

Therefore:

$$x_0 = \frac{x_t - \sigma_t \epsilon}{\alpha_t} \quad (312)$$

The model can therefore predict the noise:

$$\boxed{\epsilon_\theta(x_t, t)} \quad (313)$$

rather than directly predicting x_0 .

They define:

$$\boxed{\epsilon_\theta(x_t, t) = \frac{x_t - \alpha_t x_\theta(x_t, t)}{\sigma_t}} \quad (314)$$

This is equation (14).

67.9 Why This Works

We have:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon \quad (315)$$

Suppose the model predicts $x_\theta \approx x_0$. Then:

$$\epsilon_\theta = \frac{x_t - \alpha_t x_\theta}{\sigma_t} \quad (316)$$

Substitute x_t :

$$\epsilon_\theta = \frac{\alpha_t x_0 + \sigma_t \epsilon - \alpha_t x_\theta}{\sigma_t} \quad (317)$$

Therefore:

$$\epsilon_\theta = \epsilon + \frac{\alpha_t}{\sigma_t} (x_0 - x_\theta) \quad (318)$$

So if $x_\theta \rightarrow x_0$, then $\epsilon_\theta \rightarrow \epsilon$. Hence predicting x_0 and predicting the noise are mathematically related.

67.10 The Crucial Transformation

They show:

$$\boxed{\|x_0 - x_\theta\|^2 = \frac{\sigma_t^2}{\alpha_t^2} \|\epsilon - \epsilon_\theta\|^2} \quad (319)$$

But:

$$\frac{\sigma_t^2}{\alpha_t^2} = \frac{1}{\text{SNR}(t)} \quad (320)$$

Therefore the complicated reconstruction objective can be expressed as a noise prediction objective.

Eventually, with the appropriate reweighting, we arrive at the familiar diffusion loss:

$$\boxed{\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]} \quad (321)$$

67.11 The Whole Derivation in One Map

Clean data

`x`

`Forward diffusion`

`x = x +`

`x`

Diffusion network

(x,t)

compare with actual

$|| - ||^2$

Update neural network

And conceptually:

$$\boxed{x_0 \xrightarrow{\text{add noise}} x_t \xrightarrow{\text{network}} \hat{\epsilon}} \quad (322)$$

Training teaches:

$$\boxed{\hat{\epsilon} \approx \epsilon} \quad (323)$$

67.12 During Generation

This same learned noise predictor is used in reverse.

Start:

$$x_T \sim \mathcal{N}(0, I) \quad (324)$$

Then:

$$x_T \rightarrow x_{T-1} \rightarrow x_{T-2} \rightarrow \cdots \rightarrow x_0 \quad (325)$$

At each step:

$$\boxed{x_t \xrightarrow{\epsilon_\theta(x_t, t)} \hat{\epsilon}} \quad (326)$$

The predicted noise is used to calculate the mean of the reverse distribution and sample x_{t-1} .

So the model never directly says: "Generate this image." It repeatedly answers: "Given this noisy sample at timestep t , what noise is present?"

Removing that predicted noise step by step eventually gives the generated sample.

67.13 And for Latent Diffusion Specifically

Everything above is the same mathematics, except x is replaced by the latent representation z .

Instead of:

$$x_0 \rightarrow x_t \rightarrow x_0 \quad (327)$$

LDM does:

$$\boxed{z_0 \rightarrow z_t \rightarrow z_0} \quad (328)$$

where:

$$z_0 = E(x) \quad (329)$$

and finally:

$$\boxed{z_0 \xrightarrow{D} \hat{x}} \quad (330)$$

So the complete LDM training objective can be mentally viewed as:

$$\boxed{x \xrightarrow{E} z_0 \xrightarrow{\text{add noise}} z_t \xrightarrow{\epsilon_\theta} \hat{\epsilon} \xrightarrow{\text{MSE}} \text{update diffusion model}} \quad (331)$$

while generation is:

$$\boxed{\mathcal{N}(0, I) \xrightarrow{\text{reverse diffusion}} z_0 \xrightarrow{D} \hat{x}} \quad (332)$$

That is the connection between the ELBO derivation and the much simpler-looking noise-prediction loss you see in practical diffusion implementations.

68 KL Divergence Between Gaussians

This section proves the Gaussian KL formula used in diffusion.

68.1 Setup

Suppose we have two Gaussian distributions with the same variance:

$$q(x) = \mathcal{N}(x; \mu_1, \sigma^2) \quad (333)$$

and:

$$p(x) = \mathcal{N}(x; \mu_2, \sigma^2) \quad (334)$$

We want to calculate $D_{KL}(q||p)$.

68.2 KL Definition

$$D_{KL}(q||p) = \mathbb{E}_q \left[\log \frac{q(x)}{p(x)} \right] \quad (335)$$

68.3 Gaussian Densities

The Gaussian density is:

$$q(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x - \mu_1)^2}{2\sigma^2} \right) \quad (336)$$

and:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x - \mu_2)^2}{2\sigma^2} \right) \quad (337)$$

68.4 Take the Ratio

$$\frac{q(x)}{p(x)} = \frac{\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu_1)^2}{2\sigma^2}}}{\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu_2)^2}{2\sigma^2}}} \quad (338)$$

The normalization terms cancel because the variances are the same:

$$= \exp \left[-\frac{(x-\mu_1)^2}{2\sigma^2} + \frac{(x-\mu_2)^2}{2\sigma^2} \right] \quad (339)$$

68.5 Take the Log

$$\log \frac{q(x)}{p(x)} = \frac{(x-\mu_2)^2 - (x-\mu_1)^2}{2\sigma^2} \quad (340)$$

68.6 Take Expectation Under q

Therefore:

$$D_{KL}(q||p) = \frac{1}{2\sigma^2} E_q [(x-\mu_2)^2 - (x-\mu_1)^2] \quad (341)$$

Now notice:

- Because $x \sim q = \mathcal{N}(\mu_1, \sigma^2)$, $E_q[(x-\mu_1)^2] = \sigma^2$
- For the other term, $x - \mu_2 = (x - \mu_1) + (\mu_1 - \mu_2)$

Square it:

$$(x - \mu_2)^2 = (x - \mu_1)^2 + 2(x - \mu_1)(\mu_1 - \mu_2) + (\mu_1 - \mu_2)^2 \quad (342)$$

Taking expectation:

$$E_q[(x - \mu_2)^2] = \sigma^2 + 0 + (\mu_1 - \mu_2)^2 \quad (343)$$

The middle term is zero because $E_q[x - \mu_1] = 0$.

So:

$$E_q[(x - \mu_2)^2] = \sigma^2 + (\mu_1 - \mu_2)^2 \quad (344)$$

68.7 Substitute

Therefore:

$$D_{KL}(q||p) = \frac{1}{2\sigma^2} [\sigma^2 + (\mu_1 - \mu_2)^2 - \sigma^2] \quad (345)$$

The σ^2 terms cancel:

$$\boxed{D_{KL}(q||p) = \frac{(\mu_1 - \mu_2)^2}{2\sigma^2}} \quad (346)$$

68.8 Application to Diffusion

In diffusion we have:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \sigma_t^2 I) \quad (347)$$

and:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta, \sigma_t^2 I) \quad (348)$$

If their covariance/variance is the same, the KL becomes:

$$\boxed{D_{KL}(q||p_\theta) = \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t - \mu_\theta\|^2} \quad (349)$$

for the multidimensional case.

And because the reverse mean is parameterized using the network's x_0 prediction:

$$\tilde{\mu}_t - \mu_\theta \propto x_0 - x_\theta(x_t, t) \quad (350)$$

we eventually get:

$$\boxed{\|x_0 - x_\theta(x_t, t)\|^2} \quad (351)$$

which is the MSE-like term.

69 Sampling in LDM

Here, sampling means generating a new image (or latent) by running the learned reverse diffusion process.

69.1 During Training

The model learns how to remove noise.

You have a clean latent z_0 and progressively add noise:

$$z_0 \rightarrow z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_T \quad (352)$$

The model learns to reverse this process.

69.2 During Sampling

Now we want to actually generate an image.

We start with random Gaussian noise:

$$\boxed{z_T \sim \mathcal{N}(0, I)} \quad (353)$$

Then repeatedly denoise:

$$z_T \rightarrow z_{T-1} \rightarrow z_{T-2} \rightarrow \dots \rightarrow z_0 \quad (354)$$

At each step, the diffusion model predicts how to move from $z_t \rightarrow z_{t-1}$.

Finally:

$$\hat{x} = D(z_0) \quad (355)$$

where D is the decoder.

So:

$$\boxed{\text{Sampling} = \text{running the reverse diffusion process to generate a sample}} \quad (356)$$

69.3 In Inpainting Specifically

Suppose part of the image is known:

KNOWN

← missing

During sampling, the diffusion model generates plausible content for the missing region while using the known region as conditioning.

Conceptually:

$$\boxed{\text{noise} + \text{known image} + \text{mask} \xrightarrow{\text{iterative sampling}} \text{completed image}} \quad (357)$$

69.4 Why is It Called "Sampling"?

Because the diffusion model defines a probability distribution $p_\theta(x_0)$ over possible images.

Sampling means drawing one possible image from that learned distribution.

For example, if you inpaint a missing part of a person's shirt, there may be many plausible results:

$$\text{red shirt, blue shirt, black shirt, ...} \quad (358)$$

The diffusion model's learned distribution gives probabilities to these possibilities, and the reverse diffusion process produces one sample from that distribution.

So remember:

$$\boxed{\text{Training} = \text{learn the distribution}} \quad (359)$$

$$\boxed{\text{Sampling} = \text{generate a particular example from that distribution}} \quad (360)$$

70 Text → Tokens → Embeddings → Transformer

Your overall understanding is correct, but there is one important correction:

We don't usually say "tokenize → embed using self-attention."

Tokenization and embedding happen first; then the Transformer/self-attention processes those embeddings.

For an LDM such as Stable Diffusion, the process is roughly:

70.1 Step 1: Text → Tokens

Suppose the prompt is: "a dog running on the beach"

The tokenizer converts it into tokens:

$$[a, \text{dog}, \text{running}, \text{on}, \text{the}, \text{beach}] \quad (361)$$

More precisely, these are usually subword tokens, not necessarily complete words.

Each token is converted into an integer ID:

$$[id_1, id_2, \dots, id_L] \quad (362)$$

70.2 Step 2: Tokens → Embeddings

The token IDs are passed through a learned embedding layer:

$$[id_1, \dots, id_L] \rightarrow [e_1, e_2, \dots, e_L] \quad (363)$$

So now we have something like:

$$E \in \mathbb{R}^{L \times d} \quad (364)$$

For example, 77×768 in the original Stable Diffusion setup.

At this point, we have token embeddings, but they don't yet fully incorporate the context of the other words.

70.3 Step 3: Transformer Encoder / Self-Attention

The embeddings are then processed by a Transformer text encoder.

Self-attention allows each token to look at the other text tokens.

For example: "a red car"

The representation of "red" can interact with "car".

Mathematically:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (365)$$

and:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (366)$$

After several Transformer layers, we obtain contextual text embeddings:

$$\boxed{C \in \mathbb{R}^{L \times d}} \quad (367)$$

These are the representations that are passed to the diffusion U-Net.

70.4 Step 4: Meanwhile, the Image/Latent Goes Through Diffusion

Independently, we have the image latent.

For an image:

$$x \xrightarrow{\text{VAE encoder}} z_0 \quad (368)$$

During diffusion, noise is added:

$$z_0 \rightarrow z_t \quad (369)$$

So the U-Net receives a noisy latent:

$$\boxed{z_t} \quad (370)$$

70.5 Step 5: Now Comes Cross-Attention

This is the part you were asking about.

Inside the U-Net, we have latent features:

$$X_{\text{latent}} \quad (371)$$

We also have text embeddings:

$$C_{\text{text}} \quad (372)$$

Cross-attention connects them.

The important distinction is:

70.5.1 Query

comes from the image/latent features:

$$Q = X_{\text{latent}} W_Q \quad (373)$$

70.5.2 Key and Value

come from the text embeddings:

$$K = C_{\text{text}} W_K \quad (374)$$

$$V = C_{\text{text}} W_V \quad (375)$$

Then:

$$\boxed{\text{CrossAttention} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V} \quad (376)$$

So the latent features ask: "Which parts of the text are relevant to me?"

For example, a latent feature representing a region that is becoming a dog might attend strongly to the text token "dog".

70.6 Step 6: Then This Information is Used Inside the U-Net

So your statement "cross attention establishes relationship with latent and puts it into U-Net" is almost right, but more precisely:

$$\boxed{\text{Text encoder} \rightarrow \text{text embeddings}} \quad (377)$$

and:

$$\boxed{\text{Noisy latent} \rightarrow \text{U-Net}} \quad (378)$$

Inside the U-Net, cross-attention layers use the text embeddings to condition the latent processing.

So text isn't simply "put into the U-Net" once. Instead:

$$\boxed{\text{text embeddings are injected into multiple U-Net layers through cross-attention}} \quad (379)$$

71 Complete Conceptual Pipeline

TEXT PROMPT

Tokenizer

Token IDs

Token Embeddings

Transformer Text Encoder
(Self-Attention)

Contextual Text Embeddings

K, V

Noisy latent

U-Net

z_t

Cross-Attn Text embeddings

Denoising prediction

72 The Key Distinction

Self-attention:

$$\boxed{\text{text} \leftrightarrow \text{text}} \quad (380)$$

It establishes relationships among the text tokens.

Cross-attention:

$$\boxed{\text{latent} \leftrightarrow \text{text}} \quad (381)$$

It establishes relationships between image/latent features and text features.

So the complete conceptual pipeline is:

$$\boxed{\text{Text} \rightarrow \text{Tokenize} \rightarrow \text{Embedding} \rightarrow \text{Transformer self-attention} \rightarrow \text{Contextual text embeddings} \rightarrow \text{Cross-attention}} \quad (382)$$

while simultaneously:

$$\boxed{\text{Image} \rightarrow \text{VAE} \rightarrow z_0 \rightarrow \text{add noise} \rightarrow z_t \rightarrow \text{U-Net}} \quad (383)$$

and cross-attention is the bridge between these two streams.

73 Cross-Attention Happens Inside the U-Net

Yes. Exactly. In Latent Diffusion Models, the cross-attention happens inside the U-Net.

The flow is:

$$\boxed{\text{Text} \rightarrow \text{Tokenizer} \rightarrow \text{Text Encoder} \rightarrow \text{Text Embeddings}} \quad (384)$$

Meanwhile:

$$\boxed{\text{Image} \rightarrow \text{VAE Encoder} \rightarrow z_0 \rightarrow \text{Add Noise} \rightarrow z_t} \quad (385)$$

Then z_t enters the U-Net:

Text

Text Encoder

Text embeddings
K, V

U-Net

z_t Latent features

Cross-Attention

Text K,V

Denoised latent

Inside cross-attention:

$$Q = \text{U-Net latent features} \quad (386)$$

while:

$$K, V = \text{text embeddings} \quad (387)$$

So:

$$\text{CrossAttention}(X, C) = \text{softmax} \left(\frac{(XW_Q)(CW_K)^T}{\sqrt{d}} \right) (CW_V) \quad (388)$$

where:

- X = U-Net latent features
- C = text embeddings
- Q = generated from latent
- K, V = generated from text

74 One Important Distinction

The text encoder's self-attention is outside the U-Net.

$$\text{Self-attention} \rightarrow \text{Text Encoder} \quad (389)$$

$$\text{Cross-attention} \rightarrow \text{inside U-Net} \quad (390)$$

So you can remember:

Self-attention understands the relationship between words. Cross-attention lets the U-Net use those words to guide image generation.

75 Image-Conditioned Diffusion

Yes — if your conditioning information is an image rather than text, you can use a vision encoder to convert the image into feature tokens and then use those tokens as conditioning for the diffusion U-Net.

But there is an important distinction: you do not necessarily need a Vision Transformer (ViT). A CNN, ViT, or another vision encoder can produce the conditioning features.

75.1 Text-Conditioned Diffusion

For text:

$$\text{text} \rightarrow \text{tokenizer} \rightarrow \text{text encoder/Transformer} \rightarrow C_{\text{text}} \quad (391)$$

Then inside U-Net:

$$\boxed{Q = \text{U-Net features}, \quad K, V = C_{\text{text}}}$$
 (392)

through cross-attention.

75.2 Image-Conditioned Diffusion

For an image condition I_c :

$$I_c \rightarrow \boxed{\text{Vision Encoder}} \rightarrow C_{\text{image}} \quad (393)$$

For example, using a ViT:

$$I_c \rightarrow \text{patch embedding} \rightarrow \text{Vision Transformer} \rightarrow \boxed{C_{\text{image}}} \quad (394)$$

Then inside the diffusion U-Net:

$$\boxed{Q = \text{U-Net latent features}} \quad (395)$$

and:

$$\boxed{K, V = C_{\text{image}}} \quad (396)$$

so:

$$\boxed{\text{Cross-Attention}(\text{U-Net features}, \text{image features})} \quad (397)$$

75.3 Complete Architecture with Image Condition

CONDITION IMAGE

Vision Encoder
(ViT/CNN)

Image feature tokens

K, V

U-Net

Noisy latent Latent features
z_t

Cross-Attention Image K,V

Denoised latent

75.4 Two Common Ways to Use an Image Condition

75.4.1 1. Image as Semantic Conditioning

You want the model to understand what is in the image.

Then:

$$I_c \rightarrow \text{Vision Encoder} \rightarrow \text{tokens} \rightarrow \text{Cross-Attention} \quad (398)$$

This is analogous to text conditioning.

75.4.2 2. Image as Spatial Conditioning

For tasks such as inpainting, image-to-image translation, depth-to-image, edge-to-image, spatial information is extremely important.

Then simply converting the image into global ViT tokens may not be enough. You may instead provide spatial feature maps directly to the U-Net, concatenate them with latent features, or use architectures such as ControlNet-style conditioning.

75.5 For Your Understanding of LDM

The general pattern is:

$$\boxed{\text{Condition} \rightarrow \text{Condition Encoder} \rightarrow \text{Condition Features} \rightarrow \text{U-Net Cross-Attention}} \quad (399)$$

The condition can be:

$$\begin{aligned} &\text{text} \\ &\text{image} \\ &\text{audio} \\ &\text{segmentation map} \\ &\text{depth map} \\ &\text{other modalities} \end{aligned} \quad (400)$$

The encoder depends on the modality.

So:

$$\boxed{\text{text} \rightarrow \text{Text Transformer}} \quad (401)$$

$$\boxed{\text{image} \rightarrow \text{Vision Encoder/ViT}} \quad (402)$$

$$\boxed{\text{audio} \rightarrow \text{Audio Encoder}} \quad (403)$$

and their resulting feature representations can be used to condition the diffusion U-Net.

76 Summary of Key Equations

This section collects all the key equations from the paper.

Table 4: Key Equations in Latent Diffusion Models

Equation	Description
$x \xrightarrow{E} z \xrightarrow{D} \tilde{x}$	Autoencoder compression and reconstruction
$z_t = \sqrt{\alpha_t}z_0 + \sqrt{1 - \alpha_t}\epsilon$	Forward diffusion in latent space
$\epsilon_\theta(z_t, t)$	Noise prediction network
$\mathcal{L} = \mathbb{E}[\ \epsilon - \epsilon_\theta(z_t, t)\ ^2]$	Diffusion training objective
$\epsilon_\theta(z_t, t, \tau_\theta(y))$	Conditional diffusion objective
$\text{Attention}(Q, K, V) =$	Cross-attention mechanism
$\text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$	
Q	Cross-attention for conditioning
UNet features, $K, V =$	
text	
$z_T \sim \mathcal{N}(0, I)$	Start of generation
$z_T \rightarrow z_{T-1} \rightarrow \dots \rightarrow$	Reverse diffusion
z_0	
$x = D(z_0)$	Decode generated latent to image
$q(x_t x_0)$	Forward diffusion
$\mathcal{N}(x_t \alpha_t x_0, \sigma_t^2 I)$	
$p(x_0)$	Reverse process as integration
$\int p(x_T) \prod_{t=1}^T p(x_{t-1} x_t) dx_{1:T}$	
$D_{KL}(q p) = \frac{(\mu_1 - \mu_2)^2}{2\sigma^2}$	KL divergence between Gaussians
$\text{SNR}(t) = \frac{\alpha_t^2}{\sigma_t^2}$	Signal-to-noise ratio
$D_{KL}(q p_\theta) = \frac{1}{2\sigma_t^2} \ \tilde{\mu}_t -$	KL in diffusion
$\mu_\theta\ ^2$	
$p_\theta(x_{t-1} x_t)$	Equation (10)
$q(x_{t-1} x_t, x_\theta(x_t, t))$	
$x_t = \alpha_t x_0 + \sigma_t \epsilon$	Noisy sample
$\epsilon_\theta(x_t, t) = \frac{x_t - \alpha_t x_\theta(x_t, t)}{\sigma_t}$	Equation (14)
$\ x_0 - x_\theta\ ^2 = \frac{\sigma_{t_2}^2}{\alpha_t^2} \ \epsilon -$	Reconstruction to noise transformation
$\epsilon_\theta\ ^2$	

77 Conclusion

This comprehensive supplementary document has covered all the missing points from the LDM source material, including:

1. **The Fundamental Philosophy:** "Don't model every pixel directly" and "compress $\rightarrow$ diffuse $\rightarrow$ decode"
2. **Computational Savings:** The 48 $\times$ reduction and why it matters
3. **Compression Tradeoffs:** $f = 4, 8, 16$ and the sweet spot
4. **Perceptual and Adversarial Losses:** $\phi(x)$ and discriminator

5. **KL and VQ Regularization:** Complete with codebook $\mathcal{C}$ and stop-gradient sg
6. **Training Pipeline:** Steps 1-7 explained in detail
7. **Generation Pipeline:** Complete step-by-step
8. **Cross-Attention:** Complete derivation with Q, K, V
9. **Self-Attention vs Cross-Attention:** The $N \times M$ matrix
10. **Concatenation Alternative:** $z' = [z_t; c]$
11. **Text-to-Image Example:** "Golden retriever running on a beach at sunset"
12. **Semantic Compression:** Detailed explanation
13. **High-Resolution Generation:** 1024×1024 calculations
14. **Why Not Transformer:** $O(N^2)$ complexity
15. **Complete Mathematical Pipeline:** Stages 1-6
16. **What is Learned in Each Stage:** Table
17. **What Latent is NOT:** Distributed representation
18. **Why Decoder is Important:** Case A and Case B
19. **Near-Optimal Point:** Tradeoff explanation
20. **Why Diffusion is Suited to Latent Space:** Spatial structure preservation
21. **Why Better than VQGAN+Transformer:** Spatial latent + UNet
22. **Image-to-Image Tasks:** Inpainting, super-resolution, semantic synthesis
23. **General-Purpose Conditioning:** Cross-attention flexibility
24. **Training vs Inference:** Distinction explained
25. **Entire Paper in One Chain:** Complete mathematical flow
26. **Deepest Intuition:** Pixels as output space, not generation space
27. **Final Mental Model:** Three brains (Encoder, Diffusion, Decoder)
28. **L2 Blur Explanation:** $x_A = [0, 0, 0, 1, 1, 1]$ and $x_B = [0, 0, 1, 1, 1, 1]$
29. **Model A and Model B:** Autoencoder and Diffusion roles
30. **Semantic Mapping vs Semantic Synthesis:** $x \rightarrow y$ vs $y \rightarrow x$
31. **Complete Training and Generation Maps:** ASCII diagrams
32. **ELBO Derivation:** Equations (4), (8), (9), (10), (13), (14)
33. **Markov Relationship:** $\alpha_{t|s}$ and $\sigma_{t|s}^2$

34. **Noise Prediction Transformation:** ϵ_θ equation (14)
35. **Crucial Transformation:** $\|x_0 - x_\theta\|^2 = \frac{\sigma_t^2}{\alpha_t^2} \|\epsilon - \epsilon_\theta\|^2$
36. **SNR Relationship:** $\frac{\sigma_t^2}{\alpha_t^2} = \frac{1}{\text{SNR}(t)}$
37. **Complete Derivation Map:** ASCII diagram
38. **Text $\rightarrow$ Tokens $\rightarrow$ Embeddings $\rightarrow$ Transformer:** Complete pipeline
39. **Cross-Attention Inside U-Net:** $\text{CrossAttention}(X, C) = \text{softmax}(\frac{(XW_Q)(CW_K)^T}{\sqrt{d}})(CW_V)$
40. **Image-Conditioned Diffusion:** Vision encoder with ViT/CNN
41. **Condition Modalities:** Text, image, audio, segmentation map, depth map
42. **Encoder Depends on Modality:** Text $\rightarrow$ Text Transformer, Image $\rightarrow$ Vision Encoder/ViT, Audio $\rightarrow$ Audio Encoder

$$\boxed{\text{pixels are the output space, not necessarily the best generation space}} \quad (404)$$

$$\boxed{\text{Image} \leftrightarrow \text{Latent Representation} \leftrightarrow \text{Diffusion}} \quad (405)$$

$$\boxed{\text{Text} \rightarrow \text{Cross-Attention} \rightarrow \text{Diffusion}} \quad (406)$$